ONLINE ASSESSMENT · 21 OCT 2026

InMobi

Campus: IIT Delhi · Role: SDE

1. Questions

Pattern: 3 coding questions (21 Oct 2026).

Q1. Word Ladder with Per-Index Change Cost Coding

You are given a word catalogue `wordList`, a cost list `cost`, a `startWord` and an `endWord`. All words have the same length L, and `cost` has length L.

- In one step you change exactly one character of the current word; changing the character at index i costs `cost[i]`.
- Every word reached after a step must be in `wordList` (`startWord` need not be).
- The cost of a transformation sequence is the sum of the costs of its steps.

Return: all transformation sequences from `startWord` to `endWord` with the minimum total cost, each as the list of words `[startWord, …, endWord]`; an empty list if `endWord` cannot be reached.

EXAMPLES

```
startWord = "hit", endWord = "cog", cost = [1, 1, 1]
wordList  = ["hot","dot","dog","lot","log","cog"]
Output: [["hit","hot","dot","dog","cog"], ["hit","hot","lot","log","cog"]]     (total cost 4)

startWord = "aa", endWord = "bb", cost = [1, 10]
wordList  = ["ac","bc","ca","cb","bb"]
Both 3-step sequences reach "bb", but they cost differently:
  aa → ac → bc → bb   cost 10 + 1 + 10 = 21
  aa → ca → cb → bb   cost  1 + 10 + 1 = 12
Output: [["aa","ca","cb","bb"]]

startWord = "hit", endWord = "cog", cost = [1, 1, 1]
wordList  = ["hot","dot","dog","lot","log"]
Output: []          ("cog" is not in the catalogue)
```

CONSTRAINTS

- $1 \le L \le 5$; `endWord`, every word in `wordList` and `cost` have length L
- $1 \le$ `wordList.length` ≤ 500; all words in `wordList` are unique
- `startWord != endWord`; all words consist of lowercase English letters
- The total size of all answer sequences does not exceed 10^5

> Constraints as in LeetCode 126 (recalled as matching); bounds on `cost[i]` not recorded.

> Practice (closest — every change costs 1): LeetCode 126 — Word Ladder II

Q2. Count Ways to Select at Least k Non-Overlapping Pattern Occurrences Coding

You are given two strings, `log` of length n and `pattern` of length m, and an integer k.

- An **occurrence** of `pattern` is a **subsequence** of `log` equal to `pattern`: indices $i_1 < i_2 < \cdots < i_m$ with $\log[i_j] = \text{pattern}[j]$. It spans the interval $[i_1, i_m]$.
- Selected occurrences must not overlap: no two may share an index, and none may start inside another — each selected occurrence starts after the previous one has finished (their intervals $[i_1, i_m]$ are disjoint).
- A selection is a set of such occurrences; selections with more than k occurrences also count.

Return: the number of selections containing at least k non-overlapping occurrences.

```
int countWays(string log, string pattern, int k);
```

EXAMPLES

```
log = "abab", pattern = "ab", k = 1
occurrences: [0,1], [0,3], [2,3]
valid selections: {[0,1]}, {[0,3]}, {[2,3]}, {[0,1],[2,3]}     ([0,3] overlaps both others)
Output: 4

log = "abab", pattern = "ab", k = 2
Output: 1          only {[0,1],[2,3]}

log = "aabb", pattern = "ab", k = 1
occurrences: [0,2], [0,3], [1,2], [1,3] — every pair overlaps
Output: 4

log = "abab", pattern = "ab", k = 3
Output: 0
```

CONSTRAINTS

- $1 \le |\log| \le 200$
- $1 \le |\text{pattern}| < 20$, $|\text{pattern}| \le |\log|$
- $1 \le k \le |\log|$
- `log`, `pattern`: strings (character set not recorded)

> Practice (closest — counts subsequence occurrences only): LeetCode 115 — Distinct Subsequences

Q3. Company Audit Order with Cycle Detection Coding

You must audit N companies, numbered 0 to $N-1$. You are given M dependency pairs $[u, v]$: company u depends on company v, so v must be audited before u.

- **No cycle:** return an order in which every company is audited after all the companies it depends on (any valid order).
- **Cycle exists:** return the cycle that contains the **lowest-numbered** company lying on any cycle, written along the dependency edges ($u \to v$), starting from that company and repeating it at the end. The cycle must follow the actual edges, not sorted order.

Return: the audit order, or the cycle as described.

```
vector<int> auditOrder(vector<vector<int>>& dependencies, int N);
```

EXAMPLES

```
N = 4, dependencies = [[0,1], [1,2], [0,3]]
Output: [2, 1, 3, 0]          (any valid order)

N = 4, dependencies = [[0,1], [1,2], [2,0], [3,2]]
0 → 1 → 2 → 0 is a cycle; 3 depends on 2 but is not on it
Output: [0, 1, 2, 0]

N = 5, dependencies = [[1,2], [2,1], [3,4], [4,0], [0,3]]
cycles: 1 → 2 → 1 and 0 → 3 → 4 → 0; the lowest company on a cycle is 0
Output: [0, 3, 4, 0]
```

CONSTRAINTS

- $1 \le N < 5000$, $0 \le M < 5000$
- $0 \le u, v < N$; the graph may contain zero, one or several cycles

> Practice (closest): LeetCode 210 — Course Schedule II (order) · CSES 1678 — Round Trip II (find a directed cycle)